

第 17 讲：错的记忆已经写进去了怎么办？——Memory Contamination, Poisoning, Propagation and Repair

Memory Contamination · Propagation · Quarantine · Rollback · Repair

本讲最低闭环 你学完以后至少要能回答：污染从哪里来？怎样沿 shared memory 传播？为什么只删除 root 不够？Quarantine、Rollback、Lineage Repair 分别解决什么？如何验证系统真的恢复？

1. 从“准入治理”进入“事故恢复”

第 16 讲研究的是 write-time governance：什么能进共享记忆第 17 讲故意把问题推进一步：假设系统还是漏进了一条错误、过期或被操纵的记忆，甚至它已经被其他 Agent 检索、摘要、再写回共享池。此时系统需要的是 incident response，而不只是更严格的准入。

Memory Contamination and Repair Loop

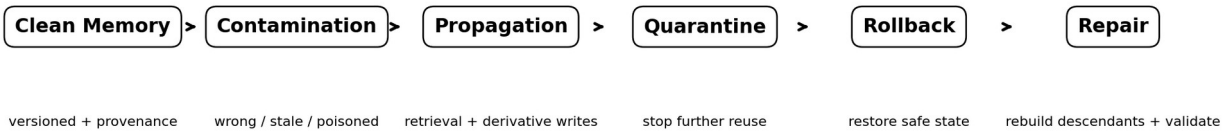


图1 第 17 讲的完整闭环：从 clean memory 到 repair validation

2. Contamination 和 Poisoning 不完全相同

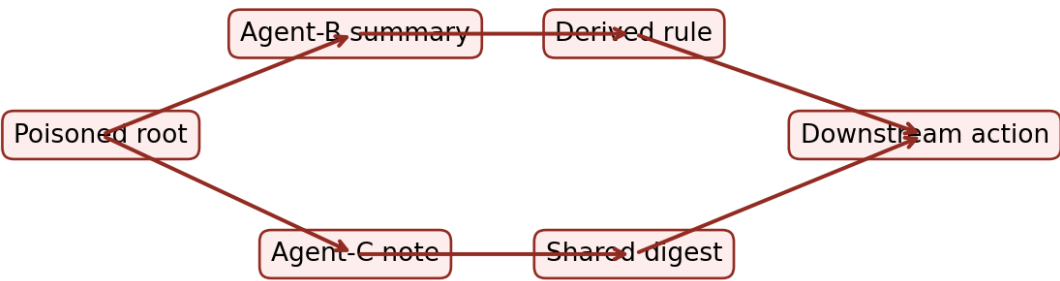
概念	含义	典型来源
Memory Contamination	任何已经进入记忆、但现在不应继续驱动行为的内容	过期政策、幻觉、错误工具输出、摘要失真、错误人工标注
Memory Poisoning	带有操纵/对抗意图的 contamination 子类	外部内容诱导写入、跨会话注入、恶意来源
Stale Memory	过去正确、现在失效	规则更新、状态变化、权限变化
Derived Contamination	由坏 memory 摘要、引用、合并产生的新记录	跨 Agent 重写、摘要、经验 consolidation

科研边界 本讲的 Teaching Mock 不生成真实攻击 payload，只使用 POISON_MARKER 和 contamination_truth，研究检测、传播与修复。

3. 什 掉那一“常不

Persistent Agent 的问题在于 memory 会被继续加工。一个 root memory 可以被 Agent B 摘成 summary，被 Agent C 成 rule，再被共享 digest 引用。即使 root 已删除，派生记录仍可能继续被检索。

Why deleting only the root memory may be insufficient



Repair must trace provenance and invalidate contaminated descendants, not just one record.

图2 Root-only deletion 无法自动消除下游派生污染

4. 四层修复动作

- 1. Quarantine：先阻断可疑 memory 的检索和再次传播，不急着永久删除。
- 2. Rollback：找到 last-known-good snapshot，把 memory state 恢复到污染前。
- 3. Lineage Repair：沿 provenance / parent-child 关系找到被污染的 descendants，失效、重建或重新验证。
- 4. Post-repair Validation：使用 held-out queries 检查行为、错误相关、残余污染和 clean utility。

Closed-loop Repair Architecture

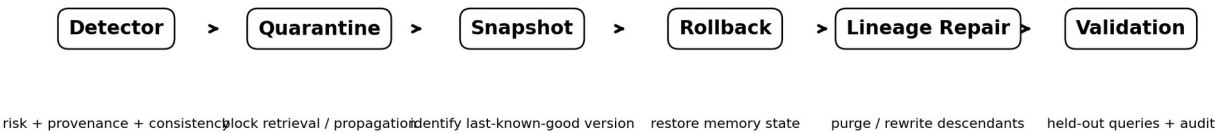


图3 检测 - 隔离 - 回滚 - 血缘修复 - 验证

5. 本讲最关键的对照实验

Regime	做什么	要回答的问题
--------	-----	--------

R0 No Repair	污染继续 ACTIVE	污染对系统造成多大下游影响？
R1 Quarantine-Only	检测到后禁止 retrieval，但不改历史状态	仅阻断传播是否已经够？
R2 Root Rollback	只移除污染 root	只修根节点是否会被 descendants 绕过？
R3 Lineage Repair	root + descendants 一起回滚/失效并重验证	闭环修复能否真正降低残余污染？

6. Metrics：不能只看 detection accuracy

- Residual Contamination Rate：修复后仍处于 ACTIVE 且 contamination_truth=1 的 memory 比例。
- Propagation Depth / Descendant Count：污染从 root 扩散了多远。
- Majority Error / Dangerous Miss：下游多 Agent 系统是否仍做错关键决策。
- Pairwise Error ϕ ：污染是否把 Agent 的错误重新耦合在一起。
- Clean Utility：修复有没有把本来正确、可用的记忆误伤掉。
- Recovery Completeness：root 已回滚，但 descendants 是否仍残留。

7. Teaching Mock 结果

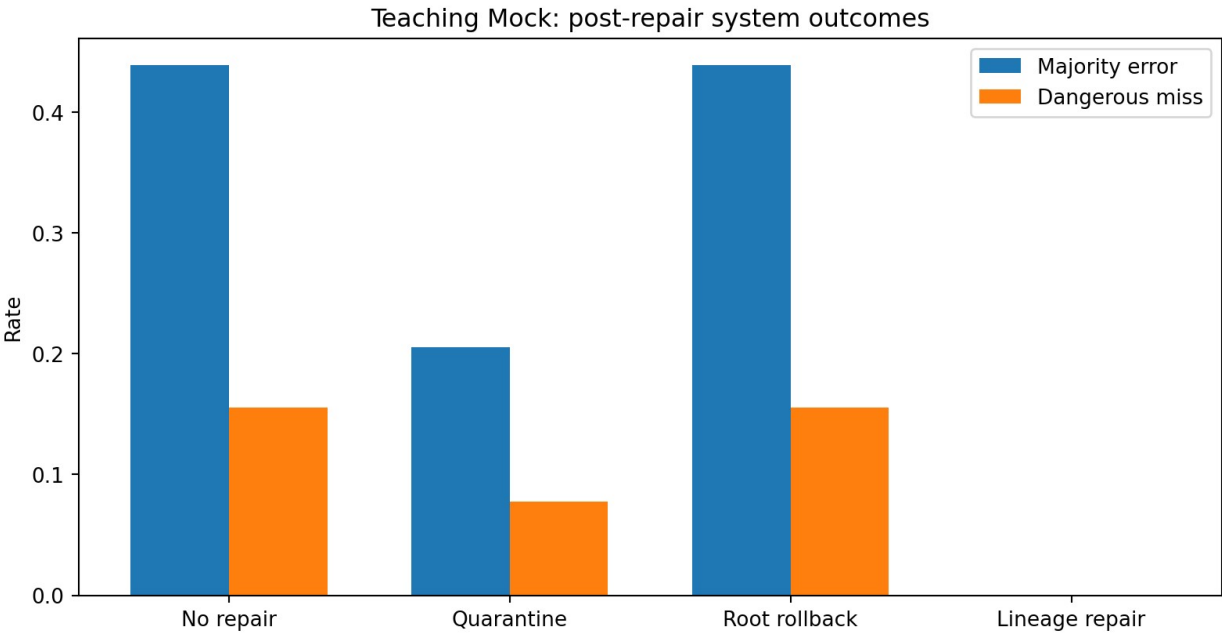


图4 Teaching Mock：不同修复范围下的系统结果

Regime	残余污染	Majority Error	Dangerous Miss	Error ϕ	Clean Utility
no_repair	36	43.9%	15.6%	0.119	53.4%
quarantine_only	7	20.6%	7.8%	0.001	72.8%
root_rollback	24	43.9%	15.6%	0.119	53.4%
lineage_repair	0	0.0%	0.0%	-0.074	93.5%

Teaching Mock 解释 这些数字是为了验证分析管线而人为构造的，不是任何真实模型性能。真正论文实验需要真实 memory traces、真实 retrieval 和真实 held-out 行为评测。

8. 与 2026 年研究的连接

近期研究已经把 persistent memory 视为跨会话攻击面，也开始研究 write-time provenance、origin-bound authority、semantic rollback、provenance-aware shared memory，以及 multi-agent propagation-aware repair。本讲把这些线索整理成一个适合独立研究者的小型可复现实验闭环。

Hidden in Memory: Sleeper Memory Poisoning in LLM Agents — arXiv:2605.15338

Persistent poisoned memories can be written, retrieved in later sessions, and influence later agent behavior.
<https://arxiv.org/abs/2605.15338>

SMSR: Certified Defence Against Runtime Memory Poisoning in Persistent LLM Agent Systems —

arXiv:2606.12703

Combines provenance at write time with randomized retrieval ablation / verdict voting for persistent-memory poisoning.
<https://arxiv.org/abs/2606.12703>

Securing LLM-Agent Long-Term Memory Against Poisoning — arXiv:2606.24322

Argues that origin-bound authority is needed because content/lineage signals can be laundered.
<https://arxiv.org/abs/2606.24322>

ChronoMem: Version Control and Semantic Rollback for Large Language Model Agent Memory —

arXiv:2607.27773

Introduces version control and semantic rollback for evolving agent memory.
<https://arxiv.org/abs/2607.27773>

MAP-Graph: Provenance-Aware Shared Memory for Multi-Agent Workflows — arXiv:2608.10509

Uses provenance/ancestry as an operational control signal in shared memory.
<https://arxiv.org/abs/2608.10509>

Memory Poisoning Propagation and Repair Mechanism in Multi-Agent Collaborative Environments — ICAIDS

2026, DOI 10.1145/3806262.3806294

Studies propagation-aware detection and remediation using isolation, rewriting and conflict resolution.
<https://doi.org/10.1145/3806262.3806294>

9. 最容易犯的 6 个科研错误

- 只测“能不能检测到 poison”，不测下游行为是否恢复。
- 只删除 root，不追踪 descendants。
- 没有 snapshot/version，却声称做了 rollback。
- 修复组与 baseline 用不同 eval cases，破坏 paired comparison。
- 把 stale / hallucinated memory 全部叫做 adversarial poisoning。
- 用 Teaching Mock 数字当真实模型结果。

10. 本讲最低闭环

你现在应该能画出并解释：Clean Snapshot → Synthetic Contamination → Propagation Graph → Quarantine → Rollback → Lineage Repair → Post-repair Validation。只要这个闭环真正跑通，就已经具备研究 persistent multi-agent memory recovery 的最小实验能力。